

Consent-Aware Runtime Mediation for Privacy in Real-Time Screen-Share AI Assistants

ANONYMOUS AUTHOR(S)

Live screen-share AI assistants can observe raw screen and speech streams, but users lack fine-grained runtime control over what the assistant may see, retain, and say. Static chatbot privacy settings and operating-system capture permissions do not address this runtime gap because sensitive content appears inside a changing multimodal stream rather than in a single user prompt. We present a runtime mediation layer that enforces consent, redaction, memory controls, and output safeguards for live multimodal assistance, interposing between synthetic capture adapters, assistant context construction, session memory, and response filtering. Using eleven synthetic screen-share scenarios, including secrets, personal data, sensitive speech fragments, screen-visible prompt injection, fast window switching, and adversarial evasion cases, we measure sensitive exposure, false blocks, task success, and latency against an unmediated baseline. The result is a bounded design and evaluation of a consent-aware runtime layer for real-time screen-share assistants, with claims limited to deterministic synthetic evidence and no use of real user, customer, or organizational data.

CCS Concepts: • **Human-centered computing** → **Interactive systems and tools**; • **Security and privacy** → *Usability in security and privacy*.

Additional Key Words and Phrases: screen-share assistants, runtime mediation, consent controls, redaction, session memory, output safeguards, synthetic benchmark

1 INTRODUCTION

Live screen-share AI assistants make an implicit exchange: the user shares a continuously changing screen or speech stream so that the assistant can help with the task at hand, while hoping that unrelated secrets, identifiers, notifications, or on-screen instructions are not exposed, retained, or repeated. That exchange is different from a conventional chatbot prompt because sensitive material can appear transiently, in another window, in a notification overlay, in speech, or in text controlled by another participant. Operating-system capture permissions and static assistant settings decide whether capture is allowed at all; they do not give the assistant a runtime boundary for what it may observe, retain, and say after capture begins.

This paper studies that boundary as a runtime mediation problem. We introduce PerceptFence, a synthetic prototype that interposes consent policy, deterministic redaction, session-memory gating, output guarding, and audit logging between screen-share inputs and assistant-facing context. The design treats screen and speech content as untrusted input, separates model context from policy-only guard context, and distinguishes three control surfaces: input exposure, retained session state, and assistant output.

The contribution is deliberately narrow. We provide (1) a threat model for screen-share runtime mediation, (2) a six-module scaffold that implements the observe/retain/say boundary, and (3) a deterministic benchmark over eleven invented scenario classes. In the current benchmark, the guarded path reduces mean sensitive exposure rate from 0.909 to 0.000 across the synthetic classes while increasing mean false block rate from 0.000 to 0.182; these numbers are evidence about this fixture set only. We do not claim deployment readiness, formal privacy, usability, legal informed consent, or protection against real-world screen-share distributions.

2 BACKGROUND AND RELATED WORK

This draft grounds the comparison axes in source-checked citations while avoiding priority claims. The related work is still a first pass; the final submission needs a deeper synthesis before claiming coverage.

Runtime permissions and capture controls. Permission systems for camera, microphone, screen capture, and application access are the closest platform analogue to this work. Prior work has studied install-time permission comprehension, user-driven access control, contextual integrity in mobile permissions, and dynamic runtime permission decisions [2, 8, 10, 11]. These systems typically authorize or deny access to a channel or resource, whereas PerceptFence assumes capture is already active and mediates the stream before assistant context, memory, or output use.

Privacy assistants and redaction tools. Prior privacy-assistant and transparency systems motivate why users need help identifying and managing sensitive material [5, 6]. Screen masking and screen-capture redaction systems are closer to PerceptFence because they operate over visible application content or captured screen material [1, 3, 7]. PerceptFence instead frames redaction as one module inside a live assistant mediation path. The deterministic redaction engine is not presented as a general detector; it is a measurable placeholder for enforcing configured policy categories on synthetic fixtures.

Prompt-injection and agent safeguards. Screen-visible instructions create a prompt-injection channel because the assistant may treat displayed text as task context. Indirect prompt-injection work shows why untrusted retrieved or observed content must not be treated as equivalent to developer or user intent [4]. Instruction-hierarchy work is relevant to this boundary because it explicitly separates higher- and lower-priority instructions inside LLM behavior [9]. This prototype does not evaluate tool use, network exfiltration, or model-internal alignment. Its output guard measures only whether literal sensitive fragments and configured indirect-reference patterns are blocked in synthetic assistant responses.

Positioning. The intended contribution is not a new capture API, a human-consent model, or a comprehensive privacy system. It is a runtime architecture and fixture-backed evaluation for separating what a live assistant may observe, retain, and say after capture begins. Any stronger related-work claim, including novelty or coverage, remains unsupported until the synthesis is complete.

3 SYSTEM MODEL

The prototype interposes capture, policy, redaction, memory, output-guard, and audit modules between synthetic screen-share inputs and assistant-facing context. The design separates what the assistant may observe, what the session may retain, and what the assistant may say. The synthetic capture adapter normalizes a fixture into a typed captured session. The policy engine maps the session to a bounded action set. The redaction engine applies deterministic transforms before assistant context construction. The session memory gate can exclude gated content from context or prune stored history after revocation. The output guard checks assistant responses using an isolated policy-only context. The audit logger records policy decisions and event types without storing raw sensitive content.

4 THREAT MODEL AND POLICY BOUNDARIES

4.1 In-Scope Adversaries

Five adversary classes are in scope for this prototype: malicious screen content, malicious meeting participants, over-privileged users, assistant output leakage, and temporal exposure during fast screen changes. These classes motivate

Table 1. Table 1: aggregate synthetic benchmark metrics across eleven scenario classes.

Metric	Baseline	Guarded	Delta
Mean sensitive exposure rate (SER)	0.9091	0.0000	-0.9091
Mean false block rate (FBR)	0.0000	0.1818	+0.1818
Mean task success rate (TSR)	0.0909	1.0000	+0.9091
Median latency across classes (ms)	0.0022	0.0343	+0.0321
p95 latency across classes (ms)	0.0024	0.0358	+0.0334
p99 latency across classes (ms)	0.0028	0.0428	+0.0400

tests for screen-visible prompt injection, sensitive notifications, credentials, personal data, spoken sensitive fragments, fast window switching, and adversarial redaction evasion.

4.2 Out-of-Scope Adversaries

Compromised infrastructure, poisoned models or software supply chains, and OS-level compromise are outside scope. The paper makes no tamper-evidence, infrastructure-hardening, model-integrity, or trusted-computing claims. Runtime mediation cannot defend against a host that has already lost control of display, microphone, clipboard, or storage boundaries.

4.3 Per-Module Boundaries

The consent engine enforces policy-mediated access control, not legal or HCI informed consent. The redaction engine reduces sensitive exposure on configured synthetic categories, not complete redaction. The memory gate controls context exclusion and history pruning in the prototype, not model-internal erasure. The output guard reduces direct leakage under the tested policy rules, not all indirect, steganographic, or future tool-mediated disclosure. The audit logger records policy decisions for review, not accountability or tamper-proof provenance.

5 EVALUATION

The evaluation uses eleven invented screen-share scenarios and compares an unmediated baseline with the guarded runtime. Metrics are sensitive exposure rate (SER), false block rate (FBR), task success rate (TSR), latency overhead, and audit-log coverage. All tables in this section are generated from the committed CSV files under eval/results/. They are synthetic diagnostics only: they are not human-subject results, deployment results, or formal privacy proofs.

6 DISCUSSION

6.1 Runtime Mediation as a Control Boundary

The runtime layer interposes at two trust boundaries: the input-to-context boundary and the output boundary. This placement allows a prototype assistant path without modifying the underlying model or operating-system capture infrastructure, but it also bounds the guarantee. The system enforces what context construction and output filtering can observe, not what a compromised host or model internals access or retain.

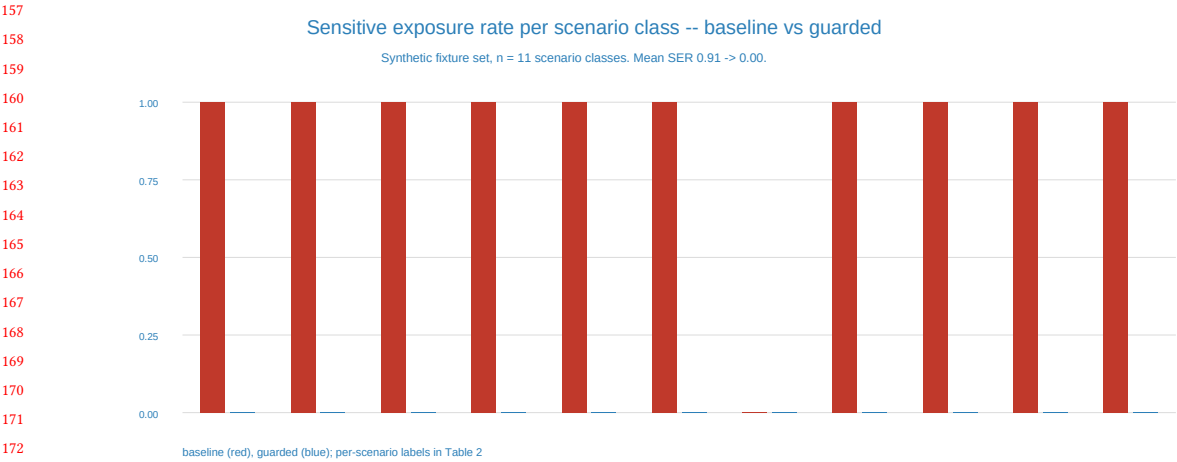


Fig. 1. Sensitive exposure rate per scenario class for the unmediated baseline and guarded runtime. The PDF was generated from docs/figures/headline_ser.svg for LaTeX inclusion.

Table 2. Table 2: per-scenario synthetic benchmark metrics.

Scenario class	Baseline SER	Guarded SER	Guarded FBR	Guarded TSR
terminal_secret	1.000	0.000	0.000	1.000
chat_notification	1.000	0.000	0.000	1.000
browser_pii	1.000	0.000	0.000	1.000
spoken_sensitive_fragment	1.000	0.000	1.000	1.000
prompt_injection_on_screen	1.000	0.000	0.000	1.000
fast_window_switching	1.000	0.000	1.000	1.000
small_font_zoomed_ui	0.000	0.000	0.000	1.000
homoglyph_credential	1.000	0.000	0.000	1.000
split_pii	1.000	0.000	0.000	1.000
mixed_sensitivity	1.000	0.000	0.000	1.000
encoded_screen_instruction	1.000	0.000	0.000	1.000

6.2 Deployment Considerations

A production deployment would require integration with operating-system screen-capture APIs, a platform hook for audio streams, a policy-management interface, and an access-control design for audit logs. The current prototype uses synthetic fixtures and does not address platform-vendor API constraints, multi-participant consent negotiation, network-level log confidentiality, or fallback behavior under distributed failure.

Table 3. Table 3: per-module ablation diagnostics.

Variant	Outcome	Ctx exp.	Out exp.	Blocks	Audit
baseline	0.000	10	10	0	0
policy_only	0.000	10	8	0	0
redaction_only	0.909	1	1	0	0
memory_gate_only	0.091	9	7	0	0
output_guard_only	0.000	10	0	10	0
audit_log_only	0.000	10	8	0	11
full_guard	1.000	0	0	5	23

6.3 Residual Risks

Residual risks include model-internal representations of content before gating, steganographic encoding in assistant output, indirect disclosure through implication or metadata, and detector misses for unseen sensitive formats. These risks are not solved by the current artifact. They are listed so that the paper does not overstate the synthetic benchmark.

7 LIMITATIONS, ETHICS, AND GENAI DISCLOSURE

7.1 Limitations

All eleven scenarios are hand-crafted fixtures representing selected risk categories. Sensitive-content detection rates, false-block rates, and task-success proxies on this benchmark may not generalize to real screen-share sessions. Tool-call escalation, multi-participant consent negotiation, cross-session consent transfer, and OS-level or infrastructure attacks are explicitly outside scope. Task-completion claims are based on synthetic task proxies, not human performance.

7.2 Ethics Note Placeholder

All data used in this artifact is synthetic. No real screen captures, personal information, credentials, customer data, organizational data, production telemetry, or human-subject study data were used. Because no human subjects were involved, IRB review is not applicable to this prototype artifact. Any future user study, public demo, or deployment evaluation requires separate ethics review and cannot reuse this placeholder as approval.

7.3 GenAI Usage Disclosure Placeholder

This section is a placeholder for the venue-specific GenAI Usage Disclosure. Before submission, the authors must state whether and how generative-AI tools were used in drafting, editing, coding, figure generation, or evaluation support, following the active IUI 2027 and ACM policies at submission time. No venue-rule claim is made here without a fresh official-source check.

8 CONCLUSION

This paper contributes a bounded design and synthetic evaluation of a consent-aware runtime mediation layer for real-time screen-share assistants. The evidence supports only synthetic-fixture claims: reduced measured sensitive exposure on the committed benchmark, with measured false-block and latency costs.

REFERENCES

- [1] Joseph Cooley and Sean Smith. 2012. *Privacy-Preserving Screen Capture: Closing the Loop for Medical Informatics Usability*. Technical Report TR2012-725. Dartmouth College Computer Science. https://digitalcommons.dartmouth.edu/cs_tr/358/
- [2] Adrienne Porter Felt, Elizabeth Ha, Serge Egelman, Ariel Haney, Erika Chin, and David Wagner. 2012. Android Permissions: User Attention, Comprehension, and Behavior. In *Proceedings of the Eighth Symposium on Usable Privacy and Security (SOUPS '12)*. ACM, 1–14. <https://doi.org/10.1145/2335356.2335360>
- [3] Abigail Goldstein, Ksenya Kveler, Tamar Domany, Igor Gokhman, Boris Rozenberg, and Ariel Farkash. 2015. Application-Screen Masking: A Hybrid Approach. *IEEE Software* 32, 4 (2015), 40–45. <https://doi.org/10.1109/MS.2015.89>
- [4] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (CCS '23)*. ACM, 79–90. <https://doi.org/10.1145/3605764.3623985>
- [5] Yuanchun Li, Fanglin Chen, Toby Jia-Jun Li, Yao Guo, Gang Huang, Matthew Fredrikson, Yuvraj Agarwal, and Jason I. Hong. 2017. PrivacyStreams: Enabling Transparency in Personal Data Processing for Mobile Apps. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies* 1, 3 (2017), 1–26. <https://doi.org/10.1145/3130941>
- [6] Bin Liu, Mads Schaarup Andersen, Florian Schaub, Hazim Almuhiemedi, Shikun Zhang, Norman Sadeh, Yuvraj Agarwal, and Alessandro Acquisti. 2016. Follow My Recommendations: A Personalized Privacy Assistant for Mobile App Permissions. In *Twelfth Symposium on Usable Privacy and Security (SOUPS 2016)*. USENIX Association, Denver, CO, 27–41. <https://www.usenix.org/conference/soups2016/technical-sessions/presentation/liu>
- [7] Sara Porat, Boaz Carmeli, Tamar Domany, Tal Drory, Amir Geva, and Abigail Tareem. 2009. Dynamic Masking of Application Displays Using OCR Technologies. *IBM Journal of Research and Development* 53, 6 (2009), 10:1–10:16. <https://doi.org/10.1147/JRD.2009.5429038>
- [8] Franziska Roesner, Tadayoshi Kohno, Alexander Moshchuk, Bryan Parno, Helen J. Wang, and Crispin Cowan. 2012. User-Driven Access Control: Rethinking Permission Granting in Modern Operating Systems. In *2012 IEEE Symposium on Security and Privacy*. IEEE, 224–238. <https://doi.org/10.1109/SP.2012.24>
- [9] Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. 2024. The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. <https://doi.org/10.48550/arXiv.2404.13208> arXiv:2404.13208 [cs.CR]
- [10] Primal Wijesekera, Arjun Baokar, Ashkan Hosseini, Serge Egelman, David Wagner, and Konstantin Beznosov. 2015. Android Permissions Remystified: A Field Study on Contextual Integrity. In *24th USENIX Security Symposium (USENIX Security 15)*. USENIX Association, Washington, D.C., 499–514. <https://www.usenix.org/conference/usenixsecurity15/technical-sessions/presentation/wijesekera>
- [11] Primal Wijesekera, Arjun Baokar, Lynn Tsai, Joel Reardon, Serge Egelman, David Wagner, and Konstantin Beznosov. 2017. The Feasibility of Dynamically Granted Permissions: Aligning Mobile Privacy with User Preferences. In *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, 1077–1093. <https://doi.org/10.1109/SP.2017.51>